CSCI 3141
Foundations of Data Science in R *(and Python)*

DALHOUSIE UNIVERSITY

Predictive Analytics (cont.)

Evaluation metrics, and the traps inside them

Prof. Gabriel Spadon, PhD

Faculty of Computer Science

spadon@dal.ca

1 The regression test set

What it is for Every regression metric below is computed on this one table, so the metrics can be compared number for number.

The data The test set is five held-out days (Table 1). The target Y is next-day peak electrical load in MW, and $\hat{y}_i$ is the model's prediction.

Table 1. Worked regression example over five held-out days

i	y_i	$\hat{y}_i$	$e_i = y_i - \hat{y}_i$	$\lvert e_i \rvert$	e_i^2	$\lvert \bar{y} - y_i \rvert$	$(\bar{y} - y_i)^2$
1	100	110	−10	10	100	20	400
2	110	100	+10	10	100	10	100
3	100	90	+10	10	100	20	400
4	120	120	0	0	0	0	0
5	170	140	+30	30	900	50	2500
Σ	600	560	+40	**60**	**1200**	**100**	**3400**

So $N_{test} = 5$ and $\bar{y} = 600/5 = 120$ MW. Each row has a role.

- Rows 1 and 2 swap the same two numbers ($y = 100, \hat{y} = 110$ and $y = 110, \hat{y} = 100$). They test MAPE.
- Rows 1 and 3 hold $y = 100$ and move the prediction up and down by 10 ($\hat{y} = 110$ and $\hat{y} = 90$). They test sMAPE.
- Row 4 is an exact prediction, so it adds a zero term to every error metric.
- Row 5 is a heat-wave day on which the model under-predicts by 30 MW. It is the largest error.

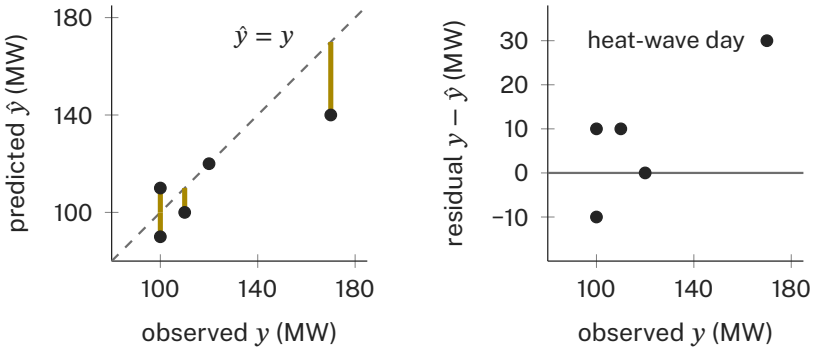


Figure 1. Predicted versus observed values and residuals for the test days

Look before you average

Draw both plots in Figure 1 before computing any metric. The predicted-versus-actual plot shows whether the predictions sit on the line $\hat{y} = y$ or on some other line. The residual plot shows whether the errors are patternless or grow with the target. Here there are four errors of at most 10 MW and one of 30 MW. Each metric below is a different answer to how much that fifth point should count.

2 Mean squared error and its root

What it is for MSE is the standard criterion for fitting a regression model. RMSE is the same quantity in the units of the target, for reporting.

The formulas

$$MSE = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} (y_i - \hat{y}_i)^2, \qquad RMSE = \sqrt{MSE},$$

where $\hat{y}_i$ is the prediction of the model under evaluation for case i and y_i is the true target value.

Worked example From the e_i^2 column of Table 1,

$$MSE = \frac{100 + 100 + 100 + 0 + 900}{5} = \frac{1200}{5} = 240 \text{ MW}^2, \qquad RMSE = \sqrt{240} = 15.49 \text{ MW}.$$

Properties

- **Units.** MSE is in the square of the target's units, MW^2 here and dollars squared for a price model. Squared units are hard to interpret operationally. RMSE restores the units of Y.
- **Outliers.** MSE is very sensitive to them because it punishes large errors more. The single 30 MW error contributes 900 of the 1200, so 20% of the cases produce 75% of the score.
- **Differentiable.** MSE is differentiable everywhere, which makes it usable as the criterion a model is fitted to optimize.
- **Same ranking.** The square root is monotone, so RMSE ranks two models on the same test set exactly as MSE does. MSE is commonly used for fitting and RMSE for reporting.

3 Mean absolute error, and choosing between MAE and RMSE

What it is for MAE reports the typical size of an error, with every unit of error counted once.

The formula

$$MAE = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} |y_i - \hat{y}_i|.$$

Worked example From the $|e_i|$ column,

$$MAE = \frac{10 + 10 + 10 + 0 + 30}{5} = \frac{60}{5} = 12 \text{ MW}.$$

Properties MAE is in the same units as Y. It is more robust to outliers than MSE, because an error of 30 counts three times an error of 10 rather than nine times. It is not differentiable at $y_i = \hat{y}_i$, which makes it harder to use as the criterion to optimize a model.

3.1 What each one answers

Which of the two to report is a published debate.[1, 11] MAE answers *on a typical day, how far off am I?* RMSE answers *how exposed am I to a bad day?* Whether sensitivity to outliers is good or bad depends on the application. If the operational cost of an error grows faster than linearly (a grid failure, an overdraft, a stock-out), sensitivity is what you want and RMSE is the right metric. If the large errors are recording mistakes or one-off events you cannot forecast, RMSE measures your data quality rather than your model.

Worked example Repair the worst day only. Move $\hat{y}_5$ from 140 to 160, so the largest error falls from 30 MW to 10 MW and every other row is untouched. Table 2 recomputes both metrics.

Table 2. MAE and RMSE before and after reducing the largest error

	before	after	change
MAE	12.00 MW	8.00 MW	-33%
RMSE	15.49 MW	8.94 MW	-42%

MAE falls by 33% and RMSE by 42%, a larger drop, because RMSE was mostly reporting that one day. Two facts make this general.

- $MAE \le RMSE \le \sqrt{N_{test}} \cdot MAE$ always. The left equality holds only when every error has the same magnitude, the right one when at most one case has nonzero error. Here $RMSE/MAE = 15.49/12 = 1.29$ sits between 1 and $\sqrt{5} = 2.24$, and its distance from 1 measures how uneven the errors are.
- MAE is minimized by predicting the conditional *median* of Y. MSE and RMSE are minimized by predicting the conditional *mean*. Choosing a metric therefore chooses which summary of the predictive distribution a fitted model will chase.

Report both. The gap between them shows how much the largest errors dominate.

R

```r
y <- c(100, 110, 100, 120, 170)
p <- c(110, 100,  90, 120, 140)
mean(abs(y - p))          # MAE   12
mean((y - p)^2)           # MSE   240
sqrt(mean((y - p)^2))     # RMSE  15.49193
library(Metrics)
mae(y, p); mse(y, p); rmse(y, p)   # same three
caret::postResample(pred = p, obs = y)
#       RMSE   Rsquared        MAE
# 15.4919334  0.7949126 12.0000000
```

Python

```python
import numpy as np
from sklearn.metrics import (mean_absolute_error,
    mean_squared_error, root_mean_squared_error)
y = np.array([100, 110, 100, 120, 170])
p = np.array([110, 100,  90, 120, 140])
mean_absolute_error(y, p)         # 12.0
mean_squared_error(y, p)          # 240.0
root_mean_squared_error(y, p)     # 15.491933
np.sqrt(np.mean((y - p) ** 2))    # 15.491933
```

Differences to know, three defaults

1. **RMSE in scikit-learn.** `mean_squared_error(y, p, squared=False)` used to be the way to get RMSE. The `squared` argument was deprecated in scikit-learn 1.4 and *removed* in 1.6. On a current install the call raises `TypeError: got an unexpected keyword argument 'squared'`. Use `root_mean_squared_error`, added in 1.4.
2. n **or** $n-1$**.** None of MSE, RMSE or MAE divides by $n-1$. They are plain means. Normalizing by a variance brings the difference back. R's `var()` divides by $n-1$ and NumPy's `np.var()` divides by n. On this data `MSE/var(y)` $= 240/850 = 0.282$ in R and `240/np.var(y)` $= 240/680 = 0.353$ in NumPy. Only the second equals the NMSE of section 4.
3. **Two different** R^2**.** `caret::postResample` prints `Rsquared` $= 0.795$ above, while `r2_score` prints 0.647 for the same vectors. Neither is a bug (section 5).

Differences to know, missing values

R propagates `NA` silently. `mean(abs(y - p))` on a vector with one missing prediction returns `NA`, not an error. scikit-learn refuses outright (`ValueError: Input contains NaN`), and NumPy returns `nan` unless you call `np.nanmean`. Decide whether a missing prediction is an error to be scored or a case to be dropped, and say which in the write-up. Dropping it changes N_{test}, and therefore every metric in this handout.

4 NMSE and NMAE

What it is for Whether 12 MW is a good error depends on the scale and difficulty of the problem. A relative metric divides the model's error by the error of a stated baseline, so it can be compared across data sets with different scales.

The formulas With a baseline that always predicts the mean $\bar{y}$,

$$NMSE = \frac{\sum_{i=1}^{N_{test}} (y_i - \hat{y}_i)^2}{\sum_{i=1}^{N_{test}} (\bar{y} - y_i)^2}, \qquad NMAE = \frac{\sum_{i=1}^{N_{test}} |y_i - \hat{y}_i|}{\sum_{i=1}^{N_{test}} |\bar{y} - y_i|}.$$

Both are unitless. Below 1 the model beats the baseline, at 1 it ties, and above 1 it loses. Both need a nonzero denominator, so a constant test target makes them undefined.

Worked example Take $\bar{y} = 120$, the mean of the test targets. The last two columns of Table 1 give the denominators.

$$NMSE = \frac{1200}{3400} = 0.353, \qquad NMAE = \frac{60}{100} = 0.600.$$

The model makes 35% of the baseline's squared error and 60% of its absolute error. The two differ for the same reason RMSE and MAE differ. The squared version is dominated by the heat-wave day, on which the baseline is also badly wrong.

4.1 Say which baseline

A training-set mean is a predictor you could actually deploy. The test-set mean is a retrospective reference, and it is the one residual R^2 uses. The two give different scores for the same predictions on the same test set. Table 3 supposes the training mean had been 130 MW.

Table 3. NMSE under the test-mean and training-mean baselines

baseline $\bar{y}$	$\sum(\bar{y} - y_i)^2$	NMSE
test mean, 120	3400	0.353
training mean, 130	3900	0.308

Neither choice is wrong. Reporting the number without saying which one you used is. The same holds for any "normalized", "relative" or "skill" score, because a normalized metric is a comparison and needs a named competitor.

5 R^2 and the squared correlation

What it is for R^2 is the most widely reported regression score. Two different quantities carry the name, and software reports both.[6]

5.1 Residual R^2, and why it can be negative

The formula With the test mean as the baseline, R^2 is the complement of NMSE.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2} = 1 - NMSE = 1 - \frac{1200}{3400} = 0.647.$$

Negative values On a test set nothing keeps R^2 above 0. A model that predicts a constant 100 MW for all five days has $SS_{res} = 0 + 100 + 0 + 400 + 4900 = 5400 > 3400$, so

$$R^2 = 1 - \frac{5400}{3400} = -0.588.$$

A negative R^2 means the model is worse than predicting the mean. It is a valid, reportable result.

5.2 Squared correlation

The formula The second quantity called "R squared" is the square of Pearson's correlation between y and $\hat{y}$.

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2} = 0.647, \qquad r^2 = \left(\frac{\sum(y_i - \bar{y})(\hat{y}_i - \bar{\hat{y}})}{\sqrt{\sum(y_i - \bar{y})^2 \ \sum(\hat{y}_i - \bar{\hat{y}})^2}} \right)^2 = 0.795.$$

Worked example Here $\bar{\hat{y}} = 112$, $\sum(y_i - \bar{y})(\hat{y}_i - \bar{\hat{y}}) = 2000$, $\sum(y_i - \bar{y})^2 = 3400$ and $\sum(\hat{y}_i - \bar{\hat{y}})^2 = 1480$, so $r = 2000/\sqrt{3400 \times 1480} = 0.8916$ and $r^2 = 0.7949$.

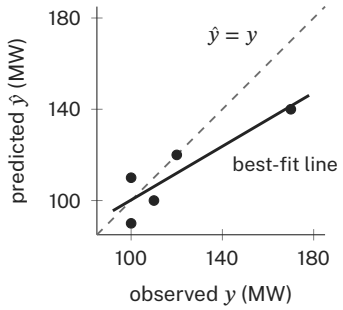


Figure 2. The test points follow a line of slope 0.59, not the line $\hat{y} = y$

Why they differ The best-fit line through the predicted-versus-actual scatter has slope 0.588 and intercept 41.41 MW, not slope 1 and intercept 0 (Figure 2). The predictions compress the range. Squared correlation measures linear association and does not change when $\hat{y}$ is shifted or rescaled, so it cannot see this bias. R^2 compares against the line $\hat{y} = y$ and does see it.

R

```r
caret::R2(p, y, formula = "traditional")
# 0.6470588   1 - SSres/SStot
caret::R2(p, y)   # 0.7949126   cor(p, y)^2,
                  # the default formula
cor(y, p)^2       # 0.7949126
1 - sum((y - p)^2) / sum((y - mean(y))^2)
# 0.6470588
```

Python

```python
from sklearn.metrics import r2_score
r2_score(y, p)
# 0.6470588   always 1 - SSres/SStot
np.corrcoef(y, p)[0, 1] ** 2     # 0.7949126
r2_score(y, np.full(5, 100))
# -0.5882353   worse than the mean
```

Differences to know

`caret::postResample` and `caret::defaultSummary` (the default regression summary in caret) report the *correlation* version. `sklearn.metrics.r2_score` reports the *residual* version and has no option to do otherwise. A model comparison table that mixes the two is not a comparison. State which definition you used, and prefer the residual one out of sample, because it is the one that detects bias.

6 MAPE

What it is for MAPE expresses each error as a fraction of the true value. The result is scale-free[4] and easy to explain to non-specialists.

The formula

$$MAPE = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} \frac{|y_i - \hat{y}_i|}{y_i}$$

Worked example The five terms are 0.100, 0.0909, 0.100, 0 and 0.1765, so

$$MAPE = \frac{0.4674}{5} = 0.0935 = 9.35\%.$$

Properties

- **Undefined if the true value is zero.** A nonzero error divided by a target approaching zero grows without bound.
- **Not differentiable** at $y_i = \hat{y}_i$. Optimizing it needs a method that handles this nonsmooth point.
- **Needs a meaningful zero.** The targets here are positive, on a scale where zero means none. A percentage error on a temperature in degrees Celsius is meaningless, because that zero is a convention.
- **Asymmetric.** Exchanging y_i and $\hat{y}_i$ gives a different MAPE. In rows 1 and 2, $y_i = 100$, $\hat{y}_i = 110$ scores 0.1, while $y_i = 110$, $\hat{y}_i = 100$ scores 0.09.
- **Ranges from 0 to ∞.** There is no upper limit when the predictions are too high, but it cannot exceed 100% for under-predictions constrained to be nonnegative.

Reading the asymmetry correctly

MAPE is asymmetric when truth and prediction exchange roles. It does not penalize over-prediction more than an equal-sized under-prediction at fixed truth.

With the true value *held fixed*, MAPE is symmetric in the sign of the error. For $y = 100$, both $\hat{y} = 110$ and $\hat{y} = 90$ score $10/100 = 0.1$, because the denominator is the truth and the truth did not change.

Swapping the *roles* of the two numbers changes the denominator. The pair $\{100, 110\}$ scores $10/100 = 0.1$ when the smaller number is the truth and $10/110 = 0.09$ when the larger one is. *For the same absolute error, MAPE is larger when the true value is smaller.*

Because $\hat{y} \geq 0$, an under-prediction has $|\hat{y} - y|/y \leq 1$, so it never costs more than 100%, while an over-prediction is unbounded. MAPE is absolute error weighted by $1/y$, so small targets count more. Optimizing it can favor lower forecasts than optimizing MAE or MSE.[8] It does not encode a specified operational cost of under-forecasting.

The zero, and what the software does with it

Add one quiet hour to the test set with $y_6 = 0$ and $\hat{y}_6 = 5$. The sixth term is $5/0$, so the metric is undefined, and direct floating-point division returns ∞. One row destroys the metric. Table 4 shows how each library reports it.

Table 4. R and Python results for MAPE with a zero denominator

call	result
`mean(abs((y - p) / y))` in R	`Inf`
`Metrics::mape(y, p)`	`Inf`
`sklearn.metrics.mean_absolute_percentage_error(y, p)`	`3.753e+15`

scikit-learn replaces y_i by $\max(|y_i|, \varepsilon)$ with $\varepsilon \approx 2.2 \times 10^{-16}$. The result is finite and enormous, with no exception and no warning, and is easy to mistake for a real score. Check for zeros in the target before you report a percentage error, in either language.

7 sMAPE

What it is for sMAPE puts the prediction into the denominator alongside the truth, to remove MAPE's role-swap asymmetry.

The formula

$$sMAPE = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} \frac{|y_i - \hat{y}_i|}{|y_i| + |\hat{y}_i|}$$

Worked example The five terms are $10/210$, $10/210$, $10/190$, 0 and $30/310$, so

$$sMAPE = \frac{0.2446}{5} = 0.0489 = 4.89\%.$$

Properties

- **Role-swap symmetric.** Exchanging y_i and $\hat{y}_i$ leaves the value unchanged. Rows 1 and 2 both score $10/210 = 4.76\%$.
- **Asymmetric in another way.** Hold the true value and the absolute deviation fixed and move in the two directions. Rows 1 and 3 give $y_i = 100$, $\hat{y}_i = 110$ at 4.76% and $y_i = 100$, $\hat{y}_i = 90$ at $10/190 = 5.26\%$. The denominator grows when the prediction is high and shrinks when it is low.[2]
- **Bounded but fragile near zero.** The value lies in $[0, 1]$. It is undefined when $y_i = \hat{y}_i = 0$ and sensitive to small changes when both are near zero.
- **Needs a meaningful zero**, as MAPE does.

Differences to know, two sMAPE conventions

The denominator above is $|y_i| + |\hat{y}_i|$, which bounds the metric in $[0, 1]$, that is 0–100%. The other widely used definition divides by $(|y_i| + |\hat{y}_i|)/2$ and therefore reports exactly *twice* the value, on a 0–200% scale.[4] R's `Metrics::smape` uses that doubled form.[3] A "sMAPE of 9.79%" and a "sMAPE of 4.89%" can describe the same predictions. Check the definition inside the function you call before comparing your number with a published one.

The code below computes the relative metrics of sections 4–7.

R

```r
sum((y - p)^2)  / sum((y - mean(y))^2)
# NMSE 0.3529412
sum(abs(y - p)) / sum(abs(y - mean(y)))
# NMAE 0.6
mape(y, p)    # 0.09347594
smape(y, p)   # 0.09785755, doubled convention
```

Python

```python
from sklearn.metrics import (
    mean_absolute_percentage_error)
1 - r2_score(y, p)   # NMSE 0.3529412
mean_absolute_percentage_error(y, p)
# 0.09347594, a fraction
# no NMAE and no sMAPE in scikit-learn
np.sum(np.abs(y - p)) / np.sum(
    np.abs(y - y.mean()))   # NMAE 0.6
np.mean(np.abs(y - p) / (np.abs(y) + np.abs(p)))
# sMAPE 0.04892877
```

Differences to know

The explicit NMSE and NMAE formulas normalize against the mean of the values passed to them. If the training mean is the intended baseline, compute and substitute it explicitly. `1 - r2_score()` always uses the test mean. `sklearn`'s MAPE returns a fraction, not a percentage, so multiply by 100 before putting it in a table.

8 Nine numbers from one test set

What it is for This section puts every regression metric side by side, so the choice between them is visible (Table 5).

Table 5. Nine regression metrics computed on the same test set

Metric	Value	Units	What it answers
MAE	12.00	MW	Typical size of an error.
MSE	240.00	MW^2	Average squared error; fitting criterion.
RMSE	15.49	MW	Typical error, with large errors weighted up.
NMSE	0.353	none	Squared error as a fraction of the baseline's.
NMAE	0.600	none	Absolute error as a fraction of the baseline's.
$R^2 = 1 - SS_{res}/SS_{tot}$	0.647	none	Improvement over the test-mean baseline; $= 1 - NMSE$ here.
r^2 (squared correlation)	0.795	none	How well predictions track, ignoring bias.
MAPE	9.35%	%	Error relative to each true value.
sMAPE	4.89%	%	Error relative to truth and prediction together.

One model and one test set give nine defensible numbers. The same five predictions can be reported as "5% error" (sMAPE), as "60% of a naïve baseline" (NMAE), as $r^2 = 0.79$ or as $R^2 = 0.65$, and all four are true. A reader told "about 5% off" and a reader told "60% of a naïve baseline" form very different impressions of the same predictions. Choose the metric before you see the results, justify it from the decision the model supports, and report it with its baseline, its units and the size of the test set.

9 Common misconceptions

- **"RMSE and MAE measure the same thing in different units."** They agree only when all errors have the same magnitude. The ratio $RMSE/MAE$ measures how uneven the errors are, and it ranges from 1 to $\sqrt{N_{test}}$.
- **"R^2 is between 0 and 1."** On a test set R^2 can be arbitrarily negative, and a negative value is a result, not a fault.
- **"R^2 is the square of the correlation."** Two different quantities carry the name. Here they differ by 0.15, and two popular libraries report different ones by default.
- **"A high correlation means good predictions."** Correlation does not change under shifts and positive rescaling, so predictions that are all 50 MW too high, or all half the right size, can still correlate at 0.99.
- **"MAPE is scale-free, so it is always safe."** It needs a strictly positive scale with a meaningful zero, it is undefined at $y = 0$, and its inverse-target weighting can favor low forecasts.

- **"sMAPE fixes MAPE's asymmetry."** It fixes the role-swap asymmetry and introduces another one, and it comes in two conventions that differ by a factor of two.
- **"NMSE below 1 means the model is good."** It means the model beat a constant baseline, on this test set, with that baseline's mean. Against a seasonal or last-value baseline the same model may lose.

10 Summary

- Each metric has its own pros and cons.
- MSE is differentiable, which makes it easier to use as a model optimization criterion.
- RMSE and MAE are expressed in the same units as the target variable.
- Relative metrics make it easier to compare results across data sets with different scales.
- Name the baseline of a normalized metric, say which R^2 your software reported, and check percentage-metric denominators for zeros.

References

[1] T. Chai and R. R. Draxler. "Root mean square error (RMSE) or mean absolute error (MAE)? Arguments against avoiding RMSE in the literature". In: *Geoscientific Model Development* 7.3 (2014), pp. 1247–1250.

[2] P. Goodwin and R. Lawton. "On the asymmetry of the symmetric MAPE". In: *International Journal of Forecasting* 15.4 (1999), pp. 405–408.

[3] B. Hamner and M. Frasco. *Metrics: Evaluation Metrics for Machine Learning*. Version 0.1.4. CRAN. 2018.

[4] R. J. Hyndman and A. B. Koehler. "Another look at measures of forecast accuracy". In: *International Journal of Forecasting* 22.4 (2006), pp. 679–688.

[5] M. Kuhn. *caret: Classification and Regression Training*. Version 7.0-1. Package documentation for model training, resampling, tuning and evaluation. CRAN. 2024.

[6] T. O. Kvålseth. "Cautionary note about R^2". In: *The American Statistician* 39.4 (1985), pp. 279–285.

[7] scikit-learn developers. *Metrics and Scoring: Quantifying the Quality of Predictions*. User guide, with `mean_absolute_error`, `root_mean_squared_error`, `r2_score` and `mean_absolute_percentage_error`. URL: `https://scikit-learn.org/stable/modules/model_evaluation.html`.

[8] C. Tofallis. "A better measure of relative prediction accuracy for model selection and model estimation". In: *Journal of the Operational Research Society* 66.8 (2015), pp. 1352–1362.

[9] L. Torgo. *Data Mining with R: Learning with Case Studies*. 2nd ed. Chapman and Hall/CRC, 2016.

[10] L. Torgo. *performanceEstimation: An Infra-Structure for Performance Estimation of Predictive Models*. Version 1.1.0. Archived on CRAN in 2025. CRAN. 2016.

[11] C. J. Willmott and K. Matsuura. "Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance". In: *Climate Research* 30 (2005), pp. 79–82.